

Report: Model Validation, Overfitting Control & Hyperparameter Tuning

This report summarizes the machine learning workflow conducted to demonstrate model validation, overfitting control, and hyperparameter tuning using a Decision Tree Regressor on the California Housing dataset.

1. Data Loading and Preparation

- **Dataset:** The California Housing dataset (`fetch_california_housing`) was loaded, consisting of 20,640 samples and 8 features. The target variable was 'MedHouseVal' (median house value).
- **Data Structure:** Features (X) were stored in a Pandas DataFrame, and the target (y) in a Pandas Series.

2. Train-Test Split

- The dataset was split into training and testing sets with an 80/20 ratio (`test_size=0.2`), ensuring that the model's performance could be evaluated on unseen data.
- `X_train` and `y_train` contained 16,512 samples, while `X_test` and `y_test` contained 4,128 samples.

3. Baseline Model Training and Evaluation

- A `DecisionTreeRegressor` was initialized with default parameters (`random_state=42`) and trained on the `X_train`, `y_train` data.
- **Performance on Test Set:**
 - **RMSE:** 0.7037
 - **R-squared:** 0.6221
- This model served as a benchmark for comparison with the hyperparameter-tuned model.

4. Cross-Validation for Robust Evaluation

- To obtain a more reliable estimate of the model's generalization performance, 5-Fold Cross-Validation (`KFold(n_splits=5, shuffle=True, random_state=42)`) was performed on the full dataset (X, y) for the baseline Decision Tree.
- **Cross-Validation Scores (Baseline Model):**

- **Mean CV RMSE:** 0.7152 (standard deviation +/- 0.0087)
 - **Mean CV R-squared:** 0.6157 (standard deviation +/- 0.0076)
- The consistent RMSE across folds suggested a stable, albeit not optimal, performance.

5. Hyperparameter Tuning with GridSearchCV

- **Objective:** To optimize the Decision Tree Regressor's performance and mitigate overfitting by systematically searching for the best combination of hyperparameters.
- **Hyperparameters Tuned:**
 - max_depth: [5, 10, 15, 20, None]
 - min_samples_split: [2, 5, 10, 20]
 - min_samples_leaf: [1, 2, 4, 8]
- **Methodology:** GridSearchCV was employed with 5-fold cross-validation (cv=kf) and optimized for 'neg_mean_squared_error' (equivalent to minimizing RMSE).
- **Results:**
 - **Best Parameters Found:** {'max_depth': 15, 'min_samples_leaf': 8, 'min_samples_split': 20}
 - **Best Cross-Validation RMSE:** 0.6220

6. Evaluation of the Tuned Model

- The best_estimator_ from GridSearchCV (the model with the optimal hyperparameters) was evaluated on the *unseen* X_test, y_test data.
- **Performance on Test Set (Tuned Model):**
 - **RMSE:** 0.6058
 - **R-squared:** 0.7200

7. Comparison: Baseline vs. Tuned Model

- A direct comparison of the test set metrics clearly showed the benefits of hyperparameter tuning:

Model	Test RMSE	Test R-squared
-------	-----------	----------------

Baseline Decision Tree	0.7037	0.6221
------------------------	--------	--------

Tuned Decision Tree	0.6058	0.7200
---------------------	--------	--------

- The tuned model achieved a significantly lower RMSE and a higher R-squared, indicating a more accurate and robust predictive capability on unseen data.
- A scatter plot of actual vs. predicted values for the tuned model visually confirmed its improved fit to the data.

8. Understanding Overfitting and Underfitting with Decision Trees

- An illustrative analysis was performed by training Decision Trees with varying max_depth values (from 1 to 30) and plotting their training and test RMSE.
- **Observations:**
 - At very low max_depth (e.g., 1-3), both training and test RMSE were high, indicating **underfitting** (the model was too simple to capture the underlying patterns).
 - As max_depth increased, training RMSE continuously decreased, while test RMSE initially decreased and then started to increase for higher depths (e.g., beyond 15-20).
 - The divergence between low training error and increasing test error for higher max_depth clearly demonstrated **overfitting** (the model became too complex and started learning noise from the training data, performing poorly on unseen data).

This comprehensive analysis highlights the importance of using techniques like train-test split, cross-validation, and hyperparameter tuning to build generalizable and well-performing machine learning models, effectively controlling for overfitting and underfitting.